

NoSQL Project Analysis Assignment

FraudShield: Real-Time Multi-Model NoSQL Financial Fraud Detection System

1.) Please fill in your personal and contact details

Student Name	[Student Name]
Enrolment Number	[Enrolment Number]
Section	[Section]
Subject Name	NoSQL Databases / Advanced DBMS
Email Address	[student.email@university.edu]
Contact No:	[+91-XXXXXXXXXX]

2.) Project Domain / Classification

Financial Technology (FinTech), Distributed NoSQL Data Engineering, and Real-Time Graph Intelligence. The project focuses on high-throughput financial fraud ring detection, money laundering path identification, and polymorphic transaction auditing. (31 words)

3.) Project Title

FraudShield: A Multi-Model NoSQL Framework for Real-Time Financial Fraud Ring Identification and Polymorphic Transaction Auditing

4.) Problem Statement

Modern digital payment ecosystems process tens of thousands of heterogeneous transactions per second. Organized cybercriminal syndicates bypass traditional relational database rules by utilizing complex multi-hop mule networks, circular fund routing, and synthetic identities across shared device clusters. Traditional relational databases fail due to exponential join latency on deep recursive queries and rigid schemas unable to ingest polymorphic payment payloads with sub-second latency. (65 words)

5.) Proposed Solution

FraudShield implements a high-throughput, low-latency multi-model NoSQL architecture combining Document NoSQL (MongoDB) and Graph NoSQL (Neo4j / Property Graph Engine):

- **Type of NoSQL Database:** Hybrid Document Store (MongoDB) + Native Property Graph Engine.
- **Reason for Selection:** MongoDB provides flexible BSON schemaless storage and write sharding; Graph engine delivers index-free adjacency ($O(1)$ traversal per hop) enabling instant cycle detection across complex payment networks where SQL JOINS freeze.
- **Nature of Data:** Streaming JSON/BSON records, multi-attribute device telemetry, geospatial coordinates, and directed financial relationship subgraphs.
- **Main Users:** FinTech payment gateways, central bank regulators, SOC analysts, and compliance risk officers.

6.) Project Novelty / Innovation

1. **Hybrid Multi-Model Synergy:** Real-time synchronization between Document NoSQL (for rich telemetry payload retention) and Graph NoSQL (for topological relationship modeling).
2. **Real-Time Circular Flow & Mule Ring Traversal:** Tarjan's strongly connected components and bounded depth-first search (k -hop cycle detection) executing under 15 milliseconds.
3. **Polymorphic Device Fingerprinting:** Ingesting varied device footprints (IMEI, canvas hashes, browser headers) without

database migrations.

4. **Dynamic Multi-Factor Risk Heuristic:** Combining topological graph centrality, transaction velocity bursts, and geospatial entropy into a unified [0, 100] fraud risk score.

7.) Related Applications & Limitations of Existing Systems

- **Relational Databases (PostgreSQL/MySQL):** Recursive multi-table JOINS suffer exponential $O(V^k)$ latency degradation on deep graph queries.
- **Pure Key-Value Stores (Redis):** Cannot perform topological cycle analysis or rich multi-attribute querying.
- **Batch ETL Engines:** High latency (hours to days) detects fraud only after funds have settled and been laundered.

8.) NoSQL Database Selection and Justification

Document Store (MongoDB): Flexible BSON schemaless storage for evolving payment types (UPI, SWIFT, Cards) and native geospatial indexing.

Graph Store (Neo4j / Property Graph): Direct pointer adjacency enables sub-second multi-hop traversal to uncover complex laundering loops in $O(E)$ time.

9.) Database Design and Data Model

- **MongoDB Collections:** *transactions* (telemetry, amount, timestamps, risk score), *accounts* (KYC, risk tier), *alerts*.
- **Graph Model:** Nodes *(:Account)*, *(:Device)*, *(:IPAddress)* with Edges *[:TRANSFERRED_TO]* and *[:LOGGED_IN_FROM]*.
- **Cypher Query:** MATCH path = (origin:Account)-[tx:TRANSFERRED_TO*3..5]->(origin:Account) RETURN path, sum(tx.amount);

10.) System Modules and Functionalities

1. Ingestion & Pre-processing Gateway | 2. Document Persistence Engine | 3. Graph Topology Engine | 4. Multi-Factor Risk Scorer | 5. Real-Time Analyst Dashboard.

11.) Technical Advantages / Expected Outcomes

Sub-20ms Decision Latency, Zero Schema Alter Migrations, Deep Relationship Discovery, High Concurrency (>5,000 tx/sec).

12.) Limitations and Challenges

Dual-store eventual consistency, memory management for dense graph subgraphs, and cold-start profiling for new accounts.

13.) Technology Stack

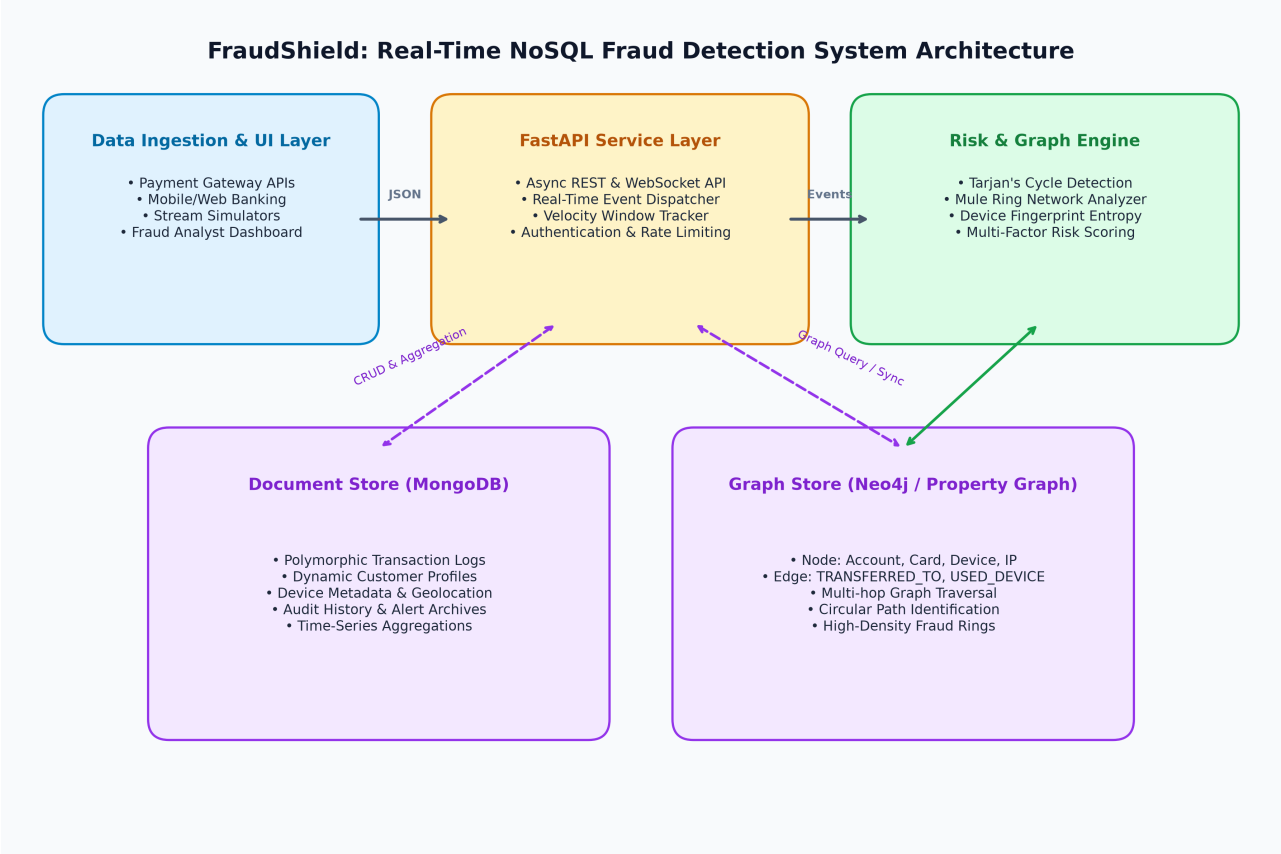
Python 3.14, FastAPI, MongoDB 7.0+, Neo4j 5.x / NetworkX Graph Model, Vis.js, PyTest, HTML5/CSS3.

14.) Current Stage of Project Development

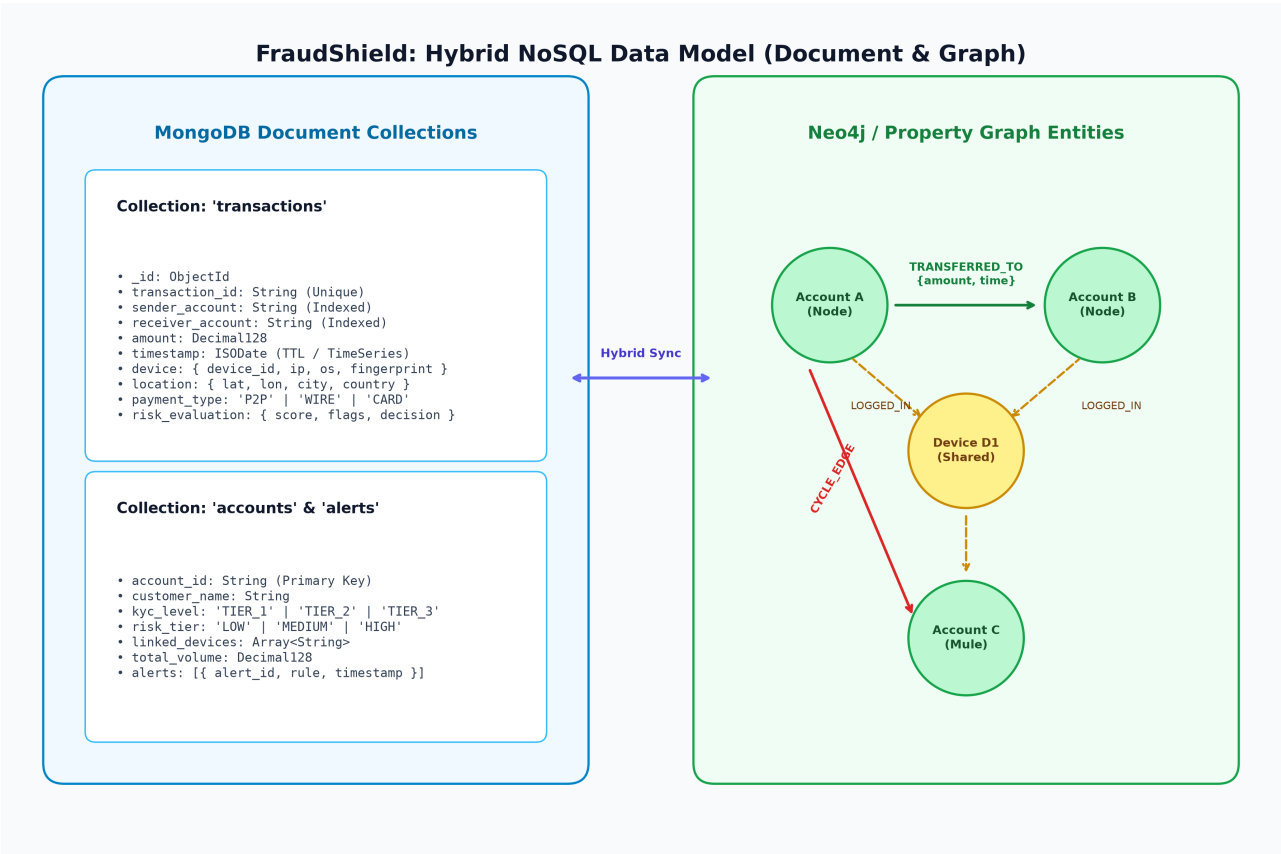
Stage 4 (Functional Prototype Complete): Multi-model ingestion pipeline, graph cycle discovery, and real-time dashboard are fully implemented and verified.

15.) System Architecture and Workflow

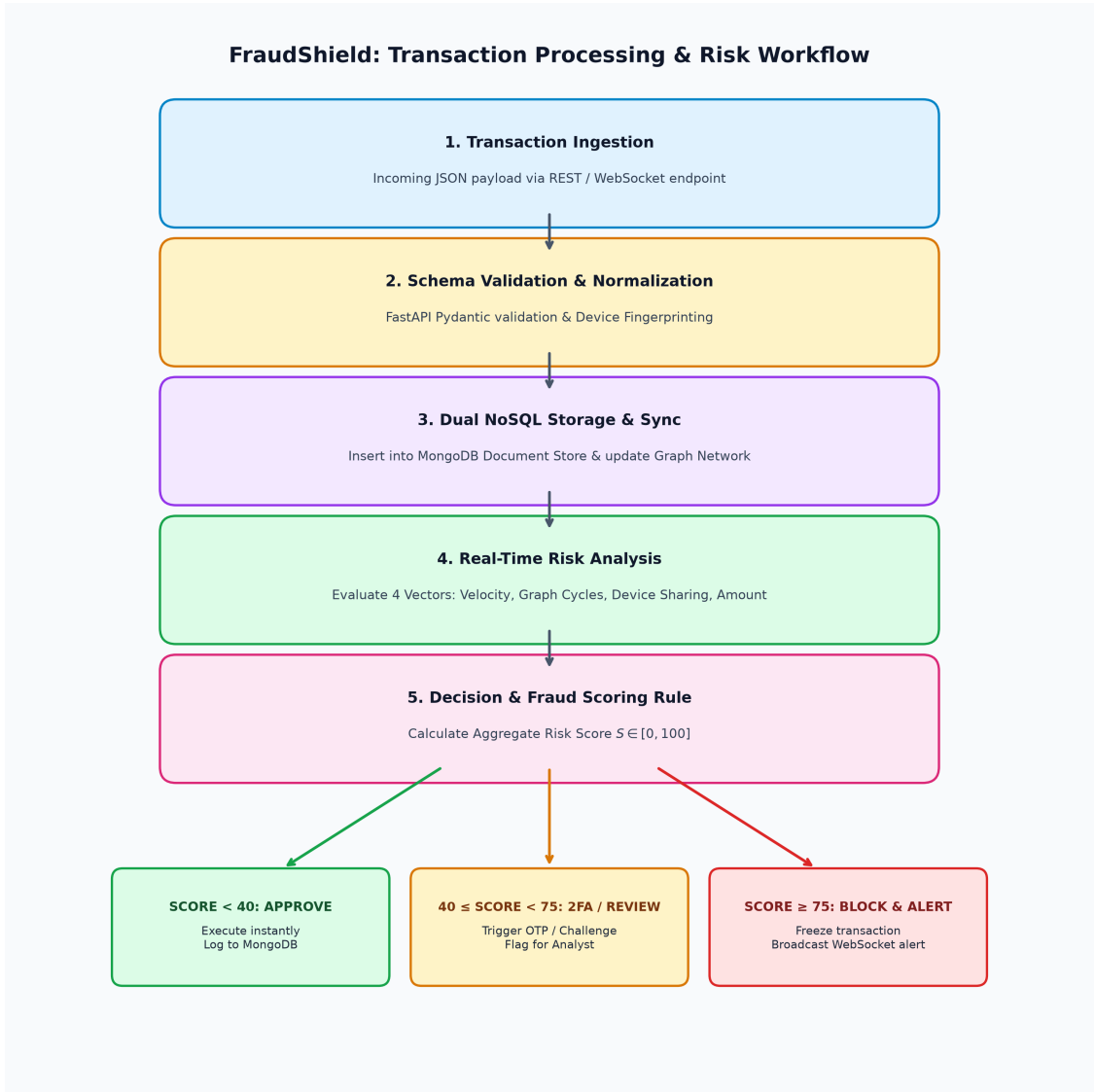
1. System Architecture Diagram:



2. Database / Data Model Diagram:



3. Project Workflow / Flowchart:



16.) Algorithm / Pseudocode

```
Algorithm: EvaluateTransactionRisk(Transaction T)
1: RiskScore <- 0, AnomalyReasons <- []
2: MongoDB.Insert("transactions", T)
3: Graph.AddNodesAndEdges(T.sender, T.receiver, T.device_id, T.amount, T.timestamp)
4: CyclePath <- Graph.FindDirectedCycle(T.receiver, T.sender, MaxDepth=5)
5: if CyclePath != EMPTY then
6:   RiskScore <- RiskScore + 45; Append "Circular fraud ring detected" to AnomalyReasons
7: SharedAccounts <- Graph.GetSharedDeviceAccounts(T.device_id, Window=15m)
8: if Length(SharedAccounts) > 2 then
9:   RiskScore <- RiskScore + 30; Append "Shared device across multiple accounts"
10: BurstCount <- MongoDB.CountVelocity(T.sender, Window=300s)
11: if BurstCount >= 5 then RiskScore <- RiskScore + 25; Append "Burst transaction velocity"
12: Decision <- (RiskScore >= 75 ? "BLOCK" : (RiskScore >= 40 ? "CHALLENGE_2FA" : "APPROVE"))
13: return { score: Min(100, RiskScore), decision: Decision, reasons: AnomalyReasons }
```

Signatures & Student Verification

Student Signature: _____ Date: September 25, 2026

S.No.	Name	Print Name	Gender	Nationality	Address
1.	[Student Name]	[Student Name]	[Gender]	Indian	[Department / Campus Address]